SERVER SCRIPT

CREATION OF PLANNING SHEET AGAIN FROM SO IF CANCEL

Script Type: API

Event Frequency: All

DocType Event: Before Insert

API Method: make_planning_sheet

Script

```
# ============================================================
# REGENERATE PLANNING SHEET API (WHITE ORDER PB ROUTING)
# ============================================================

try:
    so_name = frappe.form_dict.get("so_name")
    if not so_name: frappe.throw("Sales Order Name is required")

    existing_pl = frappe.db.get_value("Planning sheet", {"sales_order": so_name, "docstatus": ["<", 2]}, "name")
    if existing_pl:
        frappe.throw(f"⚠ An active Planning Sheet <b>{existing_pl}</b> already exists. You must Cancel it before cre
ating a new one.")

    doc = frappe.get_doc("Sales Order", so_name)

    # 1. GET UNLOCKED COLOR CHART PLAN
    cc_plan = "Default"
    try:
        raw_cc = frappe.db.get_value("DefaultValue", {
            "defkey": "production_scheduler_color_chart_plans",
            "parent": "__default"
        }, "defvalue")
        if raw_cc:
            parsed_cc = json.loads(raw_cc)
            if type(parsed_cc) == str: parsed_cc = json.loads(parsed_cc)
            if type(parsed_cc) == list:
                for plan in parsed_cc:
                    if int(plan.get("locked", 0)) == 0:
                        cc_plan = plan.get("name")
                        break
    except Exception as e:
        frappe.log_error("CC Plan Fetch Error", str(e))

    # 2. GET UNLOCKED PRODUCTION BOARD PLAN
    pb_plan = ""
    try:
        raw_pb = frappe.db.get_value("DefaultValue", {
            "defkey": "production_scheduler_production_board_plans",
```

```
        "parent": "__default"
    }, "defvalue")
    if raw_pb:
        parsed_pb = json.loads(raw_pb)
        if type(parsed_pb) == str: parsed_pb = json.loads(parsed_pb)
        if type(parsed_pb) == list:
            for plan in parsed_pb:
                if int(plan.get("locked", 0)) == 0:
                    pb_plan = plan.get("name")
                    break
except Exception as e:
    frappe.log_error("PB Plan Fetch Error", str(e))

UNIT_1_MAP = ["PREMIUM", "PLATINUM", "SUPER PLATINUM", "GOLD", "SILVER"]
UNIT_2_MAP = ["GOLD", "SILVER", "BRONZE", "CLASSIC", "SUPER CLASSIC", "LIFE STYLE",
              "ECO SPECIAL", "ECO GREEN", "SUPER ECO", "ULTRA", "DELUXE"]
UNIT_3_MAP = ["PREMIUM", "PLATINUM", "SUPER PLATINUM", "GOLD", "SILVER", "BRONZE"]
UNIT_4_MAP = ["PREMIUM", "GOLD", "SILVER", "BRONZE"]

QUAL_LIST = ["SUPER PLATINUM", "SUPER CLASSIC", "SUPER ECO", "ECO SPECIAL", "ECO GREEN",
             "ECO SPL", "LIFE STYLE", "LIFESTYLE", "PREMIUM", "PLATINUM", "CLASSIC",
             "DELUXE", "BRONZE", "SILVER", "ULTRA", "GOLD", "UV"]
QUAL_LIST.sort(key=len, reverse=True)

COL_LIST = [
    "BRIGHT WHITE", "SUPER WHITE", "MILKY WHITE", "SUNSHINE WHITE",
    "BLEACH WHITE", "WHITE MIX", "WHITE",
    "CREAM 2.0", "CREAM 3.0", "CREAM 4.0", "CREAM 5.0",
    "GOLDEN YELLOW 4.0 SPL", "GOLDEN YELLOW 1.0", "GOLDEN YELLOW 2.0",
    "GOLDEN YELLOW 3.0", "GOLDEN YELLOW",
    "LEMON YELLOW 1.0", "LEMON YELLOW 3.0", "LEMON YELLOW",
    "BRIGHT ORANGE", "DARK ORANGE", "ORANGE 2.0",
    "PINK 7.0 DARK", "PINK 6.0 DARK", "DARK PINK",
    "BABY PINK", "PINK 1.0", "PINK 2.0", "PINK 3.0", "PINK 5.0",
    "CRIMSON RED", "RED",
    "LIGHT MAROON", "DARK MAROON", "MAROON 1.0", "MAROON 2.0",
    "BLUE 13.0 INK BLUE", "BLUE 12.0 SPL NAVY BLUE", "BLUE 11.0 NAVY BLUE",
    "BLUE 8.0 DARK ROYAL BLUE", "BLUE 7.0 DARK BLUE", "BLUE 6.0 ROYAL BLUE",
    "LIGHT PEACOCK BLUE", "PEACOCK BLUE", "LIGHT MEDICAL BLUE", "MEDICAL BLUE",
    "ROYAL BLUE", "NAVY BLUE", "SKY BLUE", "LIGHT BLUE",
    "BLUE 9.0", "BLUE 4.0", "BLUE 2.0", "BLUE 1.0", "BLUE",
    "PURPLE 4.0 BLACKBERRY", "PURPLE 1.0", "PURPLE 2.0", "PURPLE 3.0", "VOILET",
    "GREEN 13.0 ARMY GREEN", "GREEN 12.0 OLIVE GREEN", "GREEN 11.0 DARK GREEN",
    "GREEN 10.0", "GREEN 9.0 BOTTLE GREEN", "GREEN 8.0 APPLE GREEN",
    "GREEN 7.0", "GREEN 6.0", "GREEN 5.0 GRASS GREEN", "GREEN 4.0",
    "GREEN 3.0 RELIANCE GREEN", "GREEN 2.0 TORQUISE GREEN", "GREEN 1.0 MINT",
    "MEDICAL GREEN", "RELIANCE GREEN", "PARROT GREEN", "GREEN"
```

```
    "MEDICAL GREEN", "RELIANCE GREEN", "PARROT GREEN", "GREEN",
    "SILVER 1.0", "SILVER 2.0", "LIGHT GREY", "DARK GREY", "GREY 1.0",
    "CHOCOLATE BROWN 2.0", "CHOCOLATE BROWN", "CHOCOLATE BLACK",
    "BROWN 3.0 DARK COFFEE", "BROWN 2.0 DARK", "BROWN 1.0",
    "CHIKOO 1.0", "CHIKOO 2.0",
    "BEIGE 1.0", "BEIGE 2.0", "BEIGE 3.0", "BEIGE 4.0", "BEIGE 5.0",
    "LIGHT BEIGE", "DARK BEIGE", "BEIGE MIX",
    "BLACK MIX", "COLOR MIX", "BLACK",
]
COL_LIST.sort(key=len, reverse=True)

WHITE_COLORS = [
    "BRIGHT WHITE", "SUPER WHITE", "MILKY WHITE", "SUNSHINE WHITE",
    "BLEACH WHITE", "WHITE MIX", "WHITE",
    "CREAM 2.0", "CREAM 3.0", "CREAM 4.0", "CREAM 5.0"
]

ps = frappe.new_doc("Planning sheet")
ps.sales_order = doc.name
ps.customer = doc.customer
ps.party_code = doc.get("party_code") or ""
ps.ordered_date = doc.transaction_date
ps.custom_planned_date = doc.delivery_date
ps.dod = doc.delivery_date
ps.planning_status = "Draft"

is_white_order = True

for it in doc.items:
    raw_txt = (it.item_code or "") + " " + (it.item_name or "")
    clean_txt = raw_txt.upper().replace("-", " ").replace("_", " ").replace("(", " ").replace(")", " ")
    clean_txt = clean_txt.replace("""", " INCH ").replace("'", " INCH ")
    words = clean_txt.split()

    gsm = 0
    for i, w in enumerate(words):
        if w == "GSM" and i > 0 and words[i-1].isdigit():
            gsm = int(words[i-1])
            break
        elif w.endswith("GSM") and w[:-3].isdigit():
            gsm = int(w[:-3])
            break

    width = 0.0
    for i, w in enumerate(words):
        if w == "W" and i < len(words)-1 and words[i+1].replace('.', '', 1).isdigit():
            width = float(words[i+1])
```

```
            break
        elif w.startswith("W") and w[1:].replace('.', '', 1).isdigit():
            width = float(w[1:])
            break
        elif w == "INCH" and i > 0 and words[i-1].replace('.', '', 1).isdigit():
            width = float(words[i-1])
            break
        elif w.endswith("INCH") and w[:-4].replace('.', '', 1).isdigit():
            width = float(w[:-4])
            break

    search_text = " " + " ".join(words) + " "
    qual = ""
    for q in QUAL_LIST:
        if (" " + q + " ") in search_text:
            qual = q
            break
    col = ""
    for c in COL_LIST:
        if (" " + c + " ") in search_text:
            col = c
            break

    if col not in WHITE_COLORS:
        is_white_order = False

    m_roll = float(it.custom_meter_per_roll or 0)
    wt = 0.0
    if gsm > 0 and width > 0 and m_roll > 0:
        wt = (gsm * width * m_roll * 0.0254) / 1000

    unit = "Unit 1"
    if qual:
        q_up = qual.upper()
        if gsm > 50 and q_up in UNIT_1_MAP: unit = "Unit 1"
        elif gsm > 20 and q_up in UNIT_2_MAP: unit = "Unit 2"
        elif gsm > 10 and q_up in UNIT_3_MAP: unit = "Unit 3"
        elif q_up in UNIT_4_MAP: unit = "Unit 4"

    ps.append("items", {
        "sales_order_item": it.name,
        "item_code": it.item_code,
        "item_name": it.item_name,
        "qty": it.qty,
        "uom": it.uom,
        "meter": float(it.custom_meter or 0),
```

```
            "meter_per_roll": m_roll,
            "no_of_rolls": float(it.custom_no_of_rolls or 0),
            "gsm": gsm,
            "width_inch": width,
            "custom_quality": qual,
            "color": col,
            "weight_per_roll": wt,
            "unit": unit,
            "party_code": ps.party_code
        })

    # Smart Routing Logic
    if is_white_order and pb_plan:
        ps.custom_plan_name = ""  # Skip Color Chart
        ps.custom_pb_plan_name = pb_plan # Send to Production Board
        assigned_plan = f"Production Board: {pb_plan}"
    else:
        ps.custom_plan_name = cc_plan # Normal Color Chart Routing
        ps.custom_pb_plan_name = ""
        assigned_plan = f"Color Chart: {cc_plan}"

    ps.insert(ignore_permissions=True)

    frappe.response["message"] = {
        "success": True,
        "pl_name": ps.name
    }

except Exception as e:
    frappe.log_error("Planning Sheet Regeneration Failed: " + str(e))
    frappe.throw("Error: " + str(e))
```

Request Limit: 5

Time Window (Seconds): 86400